

Course Outline: MCP Server — Exposing Your Tools to the World

Title: MCP Server — Exposing Your Tools to the World **Skill:** __ — Skill: Proveer de herramientas a un Agente mediante CLIs **Learnpack:** + MCP Server **File:** s__-w20d59-mcp-server **Prerequisite:** Introduction to MCPs (architecture, client-server model, JSON-RPC 2.0, tool schemas) **Target Audience:** Beginner developers who understand MCP concepts and want to implement their first MCP server in Python **Total Lessons:** 8 **Format:** Mixed (12.5% hands-on)

Course Description

You understand what MCP is — now you build one. This course walks through implementing a Python MCP server using the official Python MCP SDK. Students learn the server structure, how to define tools with JSON Schema inputs, how to handle tool calls, and how to run the server so any MCP client can connect. The hands-on challenge produces a working MCP server exposing at least one functional tool. Implementation is in Python; client connection is in the next learnpack.

Course Philosophy

This course emphasizes:

- Learn by doing — the outline stays as minimal as needed, the challenge is concrete
 - Connection to previous learning — MCP Server is the implementation of the architecture you already know
 - Incremental complexity — tool definition → tool handling → running the server
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - MCP Server Fundamentals	2
02 - Building an MCP Server	3
03 - Assessment	1
04 - Outro	1
Total	8

00.0 Exposing Your Tools to the World 

Learning Objectives:

- Explain what an MCP server does and what it exposes
- Connect MCP server concepts to the architecture from the previous learnpack

Content Outline:

- Recap: the MCP architecture has three roles — host, client, server. You're building the server.
- What an MCP server is: a program that exposes tools (and optionally resources, prompts) to any MCP-compatible client
- What you'll build in this course: a Python MCP server that exposes at least one callable tool
- The Python MCP SDK: Anthropic's official Python library that handles all the protocol mechanics (JSON-RPC 2.0, initialization handshake, tool schema validation)
- Course overview: what the server does, how to structure it, how to define tools, how to handle calls, how to run it

Transitions: This lesson sets the stage. Section 01 covers what an MCP server does and what real-world servers look like.

Key Principles:

- The Python MCP SDK handles the protocol; you focus on defining what your tools do
- An MCP server is just a Python program — it can wrap anything: CLI calls, API calls, file operations, database queries

Examples:

- An MCP server that wraps `git` CLI commands: tools like `git_status()`, `git_log()`, `git_commit(message)`
 - An MCP server for file operations: `read_file(path)`, `write_file(path, content)`, `list_directory(path)`
 - An MCP server for database queries: `run_query(sql)`, `describe_table(name)`, `list_tables()`
-

01.0 MCP Server Fundamentals

Learning Objectives:

- Describe the three things an MCP server can expose: tools, resources, prompts
- Explain the server's responsibilities during the MCP session lifecycle
- Preview what lesson 01.1 covers (use cases)

Content Outline:

- **What an MCP server exposes:**
 - **Tools:** the primary feature — callable functions with defined schemas. The server implements the business logic.
 - **Resources:** readable data (files, database records, API results) that clients can access as context
 - **Prompts:** predefined prompt templates — useful for giving agents consistent instructions for recurring tasks

- This course focuses on tools — the most common use case
- **The server's lifecycle responsibilities:**
 1. Start up and wait for a client connection
 2. Respond to the initialization handshake (send capabilities)
 3. Handle `tools/list` requests: return all tool definitions with schemas
 4. Handle `tools/call` requests: validate arguments, execute, return result
 5. Disconnect gracefully
- **What the Python SDK handles for you:** JSON-RPC message serialization, handshake protocol, schema validation (the SDK validates tool arguments against the schema before calling your code)
- Preview: 01.1 covers where MCP servers are used in practice

Transitions: You know what the server does. Lesson 01.1 shows real-world examples.

Key Principles:

- The server is the capability provider — it knows how to do things; the client just knows what's available and asks
- Schema validation is automatic: if an argument is wrong type, the SDK rejects it before your function is called

Examples: A minimal MCP server lifecycle:

```
1. Start: python my_server.py
2. Client connects via stdio
3. Client sends: initialize → server responds with capabilities
4. Client sends: tools/list → server responds with [{name: "add",
inputSchema: {...}}]
5. Client sends: tools/call {name: "add", arguments: {a: 5, b: 3}}
6. Server validates, calls your add(a, b) function, returns {content:
[{"type": "text", "text": "8"}]}
```

01.1 MCP Server Use Cases

Learning Objectives:

- Give three real-world examples of MCP servers and what tools they expose
- Explain why MCP servers are preferred over CLI wrappers when building reusable tools
- Describe the difference between a "local" MCP server (stdio) and a "remote" MCP server (SSE)

Content Outline:

- **Real-world MCP server examples:**
 - **Filesystem server (official Anthropic server):** tools for reading files, writing files, listing directories — allows Claude to access your local filesystem safely
 - **Database server:** tools for running SQL queries, describing schemas — allows AI to query databases

- **GitHub server:** tools for searching repos, reading files, creating issues — allows AI to interact with GitHub
- **Web search server:** tools for searching the web — allows AI to retrieve current information
- **Why MCP over CLI wrappers:** MCP servers add tool discovery, type safety, and interoperability. A CLI wrapper requires the agent to know the exact command; an MCP server tells the agent what's available.
- **Local vs remote servers:**
 - **Local (stdio):** runs on the same machine, communicates via stdin/stdout. Best for: development, personal tools, tools accessing local files
 - **Remote (SSE/HTTP):** runs as a web service. Best for: shared tools accessed by multiple teams, cloud-based AI services
 - This course builds a local server using stdio transport

Transitions: You understand what servers are used for. Section 02 builds one from scratch.

Key Principles:

- Most MCP servers start as local stdio servers — simple to develop, easy to test
- The MCP SDK handles both stdio and SSE transports; you switch with one line of configuration

Examples: Anthropic's official MCP servers (open source):

- [@modelcontextprotocol/server-filesystem](#) — file read/write/list
- [@modelcontextprotocol/server-github](#) — GitHub operations
- [@modelcontextprotocol/server-postgres](#) — PostgreSQL queries
- [@modelcontextprotocol/server-brave-search](#) — web search

All follow the same pattern: define tools with schemas, implement handlers, run with stdio transport.

02.0 Building an MCP Server 📖

Learning Objectives:

- Set up the Python MCP SDK in a project
- Describe the structure of an MCP server file: imports, server creation, tool registration, main entry point
- Preview what lessons 02.1 (tool definition) and 02.2 (hands-on) cover

Content Outline:

- **Installation:**

```
pip install mcp
```

- **The basic MCP server structure:**

```

from mcp.server.fastmcp import FastMCP

# Create the server
mcp = FastMCP("My Tool Server")

# Register tools using the @mcp.tool() decorator
@mcp.tool()
def my_tool(param: str) -> str:
    return f"Result: {param}"

# Run the server (stdio transport by default)
if __name__ == "__main__":
    mcp.run()

```

- **What FastMCP does:** handles all MCP protocol mechanics; you focus on defining tools
- **What @mcp.tool() does:** registers a Python function as an MCP tool; the SDK extracts the JSON schema from Python type annotations automatically
- **Type annotations → schema:** Python `str`, `int`, `float`, `bool`, `list`, `dict` map to JSON Schema types automatically; no manual schema writing needed
- Overview of 02.1 (tool definition in depth) and 02.2 (building the actual server)

Transitions: You know the structure. Lesson 02.1 goes deep into tool definition: schema, docstrings, return types.

Key Principles:

- FastMCP is the high-level SDK interface — it does the heavy lifting
- Python type annotations replace manual JSON Schema writing — FastMCP generates the schema automatically

Examples: What FastMCP generates automatically from this function:

```

@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two integers together."""
    return a + b

```

→ Tool schema:

```

{
  "name": "add",
  "description": "Add two integers together.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "a": {"type": "integer"},
      "b": {"type": "integer"}
    }
  }
}

```

```
    },
    "required": ["a", "b"]
  }
}
```

02.1 Exposing a Tool

Learning Objectives:

- Write a complete MCP tool definition with correct type annotations, docstring, and return type
- Explain how the SDK uses docstrings as the tool description
- Handle errors in a tool and return appropriate error responses

Content Outline:

- **Tool definition anatomy:**

```
@mcp.tool()
def inspect_file(path: str) -> dict:
    """
    Get information about a file.

    Args:
        path: The file path to inspect

    Returns:
        A dictionary with file size, existence, and path
    """
    if not os.path.exists(path):
        raise ValueError(f"File not found: {path}")

    return {
        "path": path,
        "exists": True,
        "size_bytes": os.path.getsize(path)
    }
```

- **The docstring:** the first line becomes the tool description in the schema; the full docstring helps the AI understand the tool
- **Return types:** tools can return `str`, `int`, `dict`, `list` — FastMCP wraps the return value in the MCP response format automatically
- **Error handling:** raise a `ValueError` or other exception — FastMCP converts exceptions to error responses that the client can parse
- **Optional parameters:** use Python default values — `def search(query: str, limit: int = 10) → limit` becomes optional in the schema
- **Complex inputs:** use `dict` for structured inputs when you need nested data

- **Tool naming:** the function name becomes the tool name; use snake_case following Python conventions

Transitions: You can define a complete tool. Lesson 02.2 puts it all together — build and run a real MCP server.

Key Principles:

- A good docstring = a good tool description = better AI tool selection
- Exceptions become error responses — raise them early and with clear messages

Anti-patterns:

- **No docstring:** the AI can't understand what the tool does without a description
- **Returning unstructured text:** prefer structured return types (dict, list) — easier for agents to parse
- **Catching all exceptions silently:** if something goes wrong, the agent should know; don't swallow errors

Examples: A complete, well-designed tool:

```
@mcp.tool()
def count_words(text: str, exclude_punctuation: bool = True) -> dict:
    """
    Count words in a text string.

    Args:
        text: The text to count words in
        exclude_punctuation: Whether to strip punctuation before counting
    (default: True)

    Returns:
        Dictionary with word count and character count
    """
    if not text.strip():
        raise ValueError("Text cannot be empty")

    if exclude_punctuation:
        import re
        text = re.sub(r'^\w\s', '', text)

    words = text.split()
    return {
        "word_count": len(words),
        "char_count": len(text)
    }
```

Challenge Prompt: Build a Python MCP server using FastMCP that exposes a tool called `inspect_file`. The tool should accept a `path` (string) argument, return a JSON object with `exists` (bool), `size_bytes` (int if exists), and `path` (string), and raise a clear error if the file doesn't exist. Run the server and verify it starts without errors.

Solution Code:

```
import os
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("File Inspector")

@mcp.tool()
def inspect_file(path: str) -> dict:
    """
    Inspect a file and return information about it.

    Args:
        path: The path to the file to inspect

    Returns:
        Dictionary with file existence, size in bytes, and path
    """
    if not os.path.exists(path):
        raise ValueError(f"File not found: {path}")

    return {
        "path": path,
        "exists": True,
        "size_bytes": os.path.getsize(path)
    }

if __name__ == "__main__":
    mcp.run()
```

Challenge Code:

```
import os
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("File Inspector")

@mcp.tool()
def inspect_file(path: str) -> dict:
    """
    Inspect a file and return information about it.

    Args:
        path: The path to the file to inspect
```



```

Returns:
    Dictionary with file existence, size in bytes, and path
"""
# Step 1: if file doesn't exist, raise ValueError with a clear message
# your code here

# Step 2: return a dict with path, exists=True, and size_bytes
# your code here
pass

if __name__ == "__main__":
    mcp.run()

```

03.0 MCP Server Assessment 🧐

Learning Objectives:

- Demonstrate understanding of MCP server implementation
- Correctly identify how FastMCP generates tool schemas from Python code
- Apply error handling patterns for MCP tools

Question Format: Multiple choice

Questions:

1. What does the `@mcp.tool()` decorator do?
 - a) It runs the tool immediately
 - b) It registers a Python function as an MCP tool and makes it discoverable by clients
 - c) It validates the tool's output schema
 - d) It connects the server to a client
2. How does FastMCP know what JSON Schema to use for a tool's inputs?
 - a) You must write the schema manually in a separate file
 - b) FastMCP reads the function's Python type annotations and converts them automatically
 - c) FastMCP uses the function's docstring to generate the schema
 - d) FastMCP requires a separate schema registration step
3. What happens if you raise a `ValueError` inside a tool function?
 - a) The server crashes
 - b) The error is silently ignored and an empty response is returned
 - c) FastMCP converts the exception to a proper MCP error response that the client can handle
 - d) The tool retries automatically
4. Which Python type annotation would you use to make an MCP tool accept a list of strings?

- a) `strings: str`
- b) `items: list[str]`
- c) `data: dict`
- d) `count: int`

5. How do you make a tool parameter optional in FastMCP?

- a) You cannot — all parameters are required
- b) Wrap it in `Optional[type]` and set a default value in the function signature
- c) Mark it with `required: False` in a schema annotation
- d) Put it in a separate `optional_params` dictionary

6. What command installs the Python MCP SDK?

- a) `npm install mcp`
- b) `conda install mcp-server`
- c) `pip install mcp`
- d) `pip install anthropic-mcp`

7. What is the role of the docstring in an `@mcp.tool()` function?

- a) It has no effect on MCP behavior
- b) It becomes the tool's description in the schema — used by AI agents to understand what the tool does
- c) It is used as the return value when the tool is called
- d) It replaces the function's type annotations

Evaluation Criteria:

- 7/7: Excellent — solid command of MCP server implementation
- 5–6/7: Good — minor gaps in schema generation or error handling
- 3–4/7: Needs review — revisit Sections 01 and 02
- Below 3/7: Restart recommended

Score Interpretation: Questions map to: `@mcp.tool` decorator (Q1), type annotation → schema (Q2), exception handling (Q3), list type annotation (Q4), optional parameters (Q5), installation (Q6), docstring role (Q7).

04.0 Your Tools Are Now MCP-Ready

Content Outline:

- Course recap: what students accomplished
 - Understood the MCP server's three capabilities: tools, resources, prompts
 - Learned the server lifecycle: initialize → list tools → handle calls → disconnect
 - Set up the Python MCP SDK and understood the FastMCP interface
 - Wrote a complete tool definition with type annotations, docstring, and error handling
 - Built and ran a working MCP server that any MCP client can connect to

- Connection to bigger picture: the next learnpack (MCP Client in TypeScript) builds the other half — a client that connects to this server, discovers its tools, and calls them
- Next steps: MCP Client learnpack — implement a TypeScript client that connects to your Python server, lists available tools, and calls `inspect_file`
- Resources for further exploration:
 - Python MCP SDK: pypi.org/project/mcp
 - MCP specification: spec.modelcontextprotocol.io
 - FastMCP examples: github.com/jlowin/fastmcp
 - Community MCP servers: github.com/modelcontextprotocol/servers
- Encouragement: you just built something that Claude, GPT-4, or any MCP-compatible AI can use — that is a real capability you now have

Note: This is conclusion only — no theory, exercises, or assessment.
